

GEM 5 Exercise #1

114501583 陳冠晰 (Vic Chen)

Acronymous and abbreviation

MPKI: Misses per 1000 instructions, which is a metric of how often the cache hits.

TLB: Translation Lookaside Buffer, which is a special high-speed cache used by the processor's MMU. Its purpose is to store recent translations of virtual memory addresses to physical addresses

ROB: Reorder Buffer, a crucial component of out-of-order execution pipelines that allows instruction to complete execution in any order but requires them to commit in their original program order.

ROI: Region of Interest

Problem 1 (Deeper CPU study)

Goal:

Study single-core CPU parameters on TimingSimpleCPU and X86O3CPU.

Methodology:

For X86O3CPU, I sweep issueWidth, numROBEntries, LQEntries, SQEntries, each varied independently in separate sweeps using makefile. The parameters will parse into gem5 scripts to configure the variables. The script constructs a SimpleBoard and a Ruby MESITwoLevelCacheHierarchy, then instantiates X86O3CPU() and sets only the knobs we are studying:

```
class MyOutOfOrderCore(BaseCPUCore):
    def __init__(self, width: int, rob_size: int, lq: int, sq: int):
        super().__init__(X86O3CPU(), ISA.X86)

        self.core.issueWidth = width
        self.core.numROBEntries = rob_size
        self.core.LQEntries = lq
        self.core.SQEntries = sq
```

Running on x86-npb-is-size-s-run workload, I experimented with the following result:

run	cpu	issueWidth	numROBEntries	LQEntries	SQEntries	simSeconds	simInsts	totalCycles	IPC	CPI	L1I_misses	L1D_misses	L2_misses	MPKI_I	MPKI_D	MPKI_L2
default						0.014884	31414282	44697084	0.7028	1.4228	1444	465372	184181	0.04597	14.8140	5.8630
o3_issue2	o3	2				0.015782	31414247	47392221	0.6629	1.5086	1432	464187	184179	0.04558	14.7763	5.8629
o3_issue4	o3	4				0.014884	31414282	44697084	0.7028	1.4228	1444	465372	184181	0.04597	14.8140	5.8630
o3_issue6	o3	6				0.01487	31414282	44655495	0.7035	1.4215	1442	465195	184151	0.0459	14.8084	5.8620
o3_lq16	o3			16		0.015145	31414303	45479871	0.6907	1.4477	1426	465303	184174	0.04539	14.8118	5.8627
o3_lq32	o3			32		0.014884	31414282	44697084	0.7028	1.4228	1444	465372	184181	0.04597	14.8140	5.8630
o3_lq64	o3			64		0.014884	31414282	44697084	0.7028	1.4228	1444	465372	184181	0.04597	14.8140	5.8630
o3_rob128	o3		128			0.014884	31414282	44697084	0.7028	1.4228	1444	465372	184181	0.04597	14.8140	5.8630
o3_rob192	o3		192			0.014394	31414306	43225804	0.7267	1.3760	1445	465328	184172	0.04600	14.8126	5.8627
o3_rob64	o3		64			0.015542	31414215	46673500	0.6731	1.4857	1418	464701	184183	0.04514	14.7927	5.8630
o3_sq16	o3				16	0.015835	31414193	47551674	0.6606	1.5137	1412	461862	184215	0.04495	14.7023	5.8641
o3_sq32	o3				32	0.015106	31414306	45364295	0.6925	1.4441	1439	462809	184205	0.04581	14.7324	5.8637
o3_sq64	o3				64	0.014884	31414282	44697084	0.7028	1.4228	1444	465372	184181	0.04597	14.8140	5.8630
timing	timing					0.047209	31414218	141769416	0.2216	4.5129	1195	460104	183835	0.03804	14.6463	5.8520

Here I use scripts to parse data for above CSV metrics generation:

- simSecond, simInsts, totalCycles: directly from stats.txt
- IPC/CPI: $IPC = \text{simInsts}/\text{cycle}$, $CPI = 1/IPC$
- L1I/D and L2 misses: `board.cache_hierarchy.[L#]-cache-*/overallMisses::total`
- MPKI: $MPKI = \text{misses}/(\text{simInsts}/1000)$

Experimental Result:

Which parameter appeared to have the greatest performance impact?

I think increasing the load queue(LQEntries) gave the largest IPC gain and ROI time reduction; ROB size helped but with diminishing returns; SQEntries had little effect; issueWidth changes were marginal.

Why is the IS benchmark (integer sort) more sensitive to memory/cache behavior than to the core pipeline width?

IS has irregular, low-locality accesses, so time is spent waiting on cache & memory. Wider issue can't help if few instructions are ready; more LQ lets the core overlap more outstanding misses.

Link your explanation to whether the benchmark is memory-bound or computebound.

Based on my results, are memory-bound, the performance improves mainly by increasing memory concurrency, not by adding execution width.

Problem 2 (Cache sensitivity, Classic vs Ruby)

Goal:

Compare cache behavior while varying only one parameter at a time – either L2 size or L2 associativity under both Ruby MESI two-level and Classic hierarchies. Workload is x86-npb-is-size-s to isolate cache effects without coherence traffic.

Methodology:

Here I compare Ruby and Classic cache hierarchies:

```
cache_hierarchy = MESITwoLevelCacheHierarchy(**cache_kwargs)
```

```
cache_hierarchy = PrivateL1SharedL2CacheHierarchy(**cache_kwargs)
```

With variables (L2_size, L2_assoc) swept by makefile parsing parameters into it. For fairness controls, I use same ISA, benchmark, and memory. The initial version of exercises required us to capture the Fwd_GETS/GETX for Ruby, but with single L1, the directory never needs to forward a request to another L1, so Fwd_* counters stay zero and many Ruby distributions are simply not emitted. Fwd_GETS/GETX happens when one core requests a block that another core currently owns and shares, prompting the directory to forward that request to the peer L1.

 陳冠晰 3 樓

5 天前 ✎ ✕

Dear TA,

I'm running Problem 2 with TimingSimpleCPU on X86 and the Ruby MESI two-level hierarchy. The stats.txt files don't contain the Fwd_GETS/Fwd_GETX. What I see in stats.txt are L2 controller events such as L1_GETS/L1_GETX, but no Fwd_GETS or Fwd_GETX:

board.cache_hierarchy.ruby_system.L2Cache_Controller.L1_GETS	152442	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.L1_GETX	308546	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.L1_PUTX	458518	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.L2_Replacement	61474	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.L2_Replacement_clean	140408	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.Mem_Data	203930	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.Mem_Ack	201882	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.WB_Data	1878	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.Ack_all	1270	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.Exclusive_Unblock	460988	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.NP.L1_GET_INSTR	1274	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.NP.L1_GETS	141301	0.00%	0.00% (Unspecified)
board.cache_hierarchy.ruby_system.L2Cache_Controller.NP.L1_GETX	61355	0.00%	0.00% (Unspecified)

In contrast, the bootcamp Ruby example here shows Fwd_GETS/GETX in its output: [run.py](#)

Is this because the NPB-IS workload is single-threaded, so with one core, there's no cross-core sharing and the directory never needs to forward (hence no Fwd_*)?

I want to make sure I'm measuring the right thing, thanks!

Sincerely,
陳冠晰

Following are the CSV metrics generated by parsing data from stats.txt:

config	simInsts/Ops	cycles	simSeconds	hostSeconds	IPC	CPI	L1I_accesses	L1I_misses	L1I_MPKI	L1D_accesses	L1D_misses	L1D_MPKI	L1_total_MPKI	L2_accesses	L2_misses	L2_MPKI
classic_L2_128kB	31414193	66558909	0.022164	40.38	0.471976	2.118751	42238379	890	0.02833	10374598	300714	9.5726	9.6009	448654	71847	2.2871
classic_L2_1MB	31414313	57380752	0.019108	38.92	0.547472	1.826579	42238539	894	0.0285	10374648	299420	9.5313	9.5598	448659	5778	0.1839
classic_L2_256kB	31414195	66336000	0.02209	39.95	0.473562	2.111655	42238382	890	0.0283	10374598	300782	9.5747	9.6030	448653	72342	2.3028
classic_L2_512kB	31414319	58694988	0.019545	38.83	0.535213	1.868414	42238551	895	0.0285	10374648	300284	9.5588	9.5873	448657	17724	0.5642
classic_L2_a16	31414195	66336000	0.02209	39.91	0.473562	2.111655	42238382	890	0.0283	10374598	300782	9.5747	9.6030	448653	72342	2.3028
classic_L2_a32	31414182	66370813	0.022101	39.72	0.473314	2.112764	42238359	886	0.0282	10374596	300805	9.5755	9.6037	448650	72412	2.3051
classic_L2_a8	31414198	66323104	0.022086	39.8	0.473654	2.111245	42238384	890	0.0283	10374602	300794	9.5751	9.6034	448653	73128	2.3279
ruby_L2_128kB	31414200	144407596	0.048088	87.02	0.217539	4.596886	42238387	1307	0.0416	10374742	460988	14.6745	14.7161	462295	203930	6.4917
ruby_L2_1MB	31414193	113682826	0.037856	80.65	0.276332	3.618834	42238385	909	0.0289	10374737	459570	14.6294	14.6583	460479	9609	0.3059
ruby_L2_256kB	31414218	141769416	0.047209	90.85	0.221587	4.512903	42238415	1195	0.038	10374748	460104	14.6463	14.6844	461299	183835	5.8520
ruby_L2_512kB	31414188	115821273	0.038568	84.51	0.27123	3.686907	42238375	1105	0.0352	10374737	459596	14.6302	14.6654	460701	22636	0.7206
ruby_L2_a16	31414218	141769416	0.047209	84.44	0.221587	4.512903	42238415	1195	0.038	10374748	460104	14.6463	14.6844	461299	183835	5.8520
ruby_L2_a32	31414216	141891291	0.04725	83.28	0.221397	4.516783	42238412	1186	0.0378	10374748	460078	14.6455	14.6833	461264	184854	5.8844
ruby_L2_a8	31414184	141484984	0.047114	85.5	0.222032	4.503853	42238368	1185	0.0377	10374735	460135	14.6474	14.6851	461320	181952	5.792

The naming of L1I, L1D, L2 in classic cache hierarchy is different from Ruby:

- L1I/D and L2 misses: board.cache_hierarchy.[l#]caches.demandMisses::total & board.cache_hierarchy.[l#]caches.overallMisses::total
- L1I/D and L2 accesses: board.cache_hierarchy.[l#]caches.demandAccesses::total & board.cache_hierarchy.[l#]caches.overallAccesses::total

The rest are just the same.

Experimental Result:

With a single core and IS workload, Classic is equal or slightly faster than Ruby. Ruby adds controller and message handling overhead that doesn't pay off when there's no coherence traffic. The cache size helps a little, but associativity doesn't. Making L2 bigger gives a MPKI drop and a small IPC gain, but with higher L2 associativity barely moves the needle. Ruby's detailed network and coherence modeling can expose gains from banking and protocol effects. In single-core IS, those advantages aren't exercised (no Fwd_* traffic, no directory forwards), so Classic's simpler path wins on overhead.

To explain why IS behaves this way, from Problem 1 we know that IS is memory-bound, not compute-bound. It has low locality with scattered accesses, with stalls come from memory latency, not core throughput. The larger caches only help if they catch a bit more reuse. For IS, the benefit is limited. With no coherence activity, Ruby's extra protocol is overhead, while Classic's leaner path works better to a single-core, memory bound workload.

Problem 3 (In-depth study of memory system)

Goal:

Quantify achieved bandwidth and average memory latency under synthetic loads. Here I added p3_3 to compare SimpleMemory vs DDR4 at the same size/bandwidth and sweep bandwidth and reading rate to study their impact.

Methodology:

Since the suggested starting point [run-mem.py](#) has no cache, the AssertionError may come from TestBoard._connect_things: with dual-channel DDR4, our generator has 1 master port while the memory exposes 2 slave ports, and with cache_hierarchy=None there's nothing to fan-out the connection. Here I added a tiny interconnect using the NoCache hierarchy. It keeps the no-cache behavior but provides a system crossbar so one generator can talk to multi-channel memory:

```
board = TestBoard(clk_freq="1GHz", generator=generator, memory=memory, cache_hierarchy=NoCache())
```

The Problem requires to use RandomGenerator as traffic generator, its --bandwidth and --read_percentage are configured by makefile parsing parameters. The memory configs include SingleChannelDDR4_2400 in p3_1, DualChannelDDR4_2400 in p3_2, and SingleChannelSimpleMemory in addition to p3_3 for SimpleMemory vs DDR4 comparison.

Following are the CSV metrics generated by parsing data from stats.txt:

config	simSeconds	hostSeconds	bytesRead	bytesWritten	bytesTotal	throughput_Bps	avgMemAccLat_ticks	avgMemAccLat_ns
cmp_ddr4	0.001	1.41	11761984	3078208	14840192	14840192000	347891.93	347.89193
cmp_ddr4_bw16GiBs	0.001	1.97	11724800	3067072	14791872	14791872000	328082.13	328.08213
cmp_ddr4_bw32GiBs	0.001	1.85	11761984	3078208	14840192	14840192000	347891.93	347.89193
cmp_ddr4_r100	0.001	1.26	14534400	0	14534400	14534400000	281725.47	281.72547
cmp_ddr4_r50	0.001	1.76	6895488	7026432	13921920	13921920000	320521.35	320.52135
cmp_simple	0.001	0.98	25373120	6624704	31997824	31997824000		
cmp_simple_bw16GiBs	0.001	0.52	13623552	3556480	17180032	17180032000		
cmp_simple_bw32GiBs	0.001	0.99	25373120	6624704	31997824	31997824000		
cmp_simple_r100	0.001	0.66	31997760	0	31997760	31997760000		
cmp_simple_r50	0.001	1.29	15869056	16128768	31997824	31997824000		
dual	0.001	3.1	22911680	5993984	28905664	28905664000	276859.45	276.85945
single	0.001	1.6	11761984	3078208	14840192	14840192000	347891.93	347.89193

- bytesRead: board.processor.cores.generator.bytesRead
- bytesWritten: board.processor.cores.generator.bytesWritten
- avgMemAccLat: board.memory.mem_ctrl0.dram.avgMemAccLat &
board.memory.mem_ctrl1.dram.avgMemAccLat
- hostSeconds: also recorded to compare wall-clock across channels/mem types

Experimental Result:

The dual-channel DDR helps under high request rates. When the generator rate (working pressure) is high enough to saturate one channel, a second channel nearly doubles the sustained throughput, because traffic is split across two independent DRAM paths. If the request rate is low, one channel isn't the bottleneck, dual-channel then show minimal bandwidth change and similar latency. For the comparison of DDR4 and SimpleMemory with same configuration in size and bandwidth, the SimpleMemory has a fixed latency model, so it looks flat and optimistic: low, steady latency and close-to-configured bandwidth as load rises; the DDR4 is timing-accurate: at low load, latency is fine, but bank conflicts & write drains raise latency and throughput at high load. We can see that the bandwidth sustained throughput scales through 16GB to 32GB until the next bottleneck (read rate). For 50% vs 100% reading rate, controllers typically prioritize the reads and buffer writes. With 100% reads, we usually see slightly lower average latency and higher effective reading bandwidth. With 50% reads, periodic writes stall reads, so latency rises and read throughput decreases, even if total bytes/sec looks similar.

Problem 4 (Full-System boot, KVM fast-forward, CPU switch)

Goal:

Demonstrate the wall-clock speedup from KVM fast-forward and collect ROI micro-architectural stats on detailed CPU(Timing / O3).

Methodology:

For method 1, system boot with KVM at the initial. And at the 2nd m5_exit, do m5.stats.dump() with m5.stats.reset() to mark ROI start. Then, switch to the detailed

CPU to run till the 3rd m5_exit (ROI end). Method 2 stays on KVM until workload ends. It will report wall-clock only, while micro-architecture counters are not modeled by KVM. Here I use X86board with Classic cache hierarchy and SingleChannelDDR3 referred to suggested base script [x86-fs-kvm-run.py](#). The workload is x86-ubuntu-24.04-boot-with-systemd which emits the 3 ROI markers.

For processor switching, SimpleProcessor can't be switched at runtime, following is the error asserted:

AttributeError: object 'SimpleProcessor' has no attribute 'switch'

Instead, I used SimpleSwitchableProcessor:

```
processor = SimpleSwitchableProcessor(
    starting_core_type=CPUtypes.KVM,
    switch_core_type=switch_to,
    isa=ISA.X86,
    num_cores=args.cores,
)
```

Following are the CSV metrics generated by parsing data from stats.txt:

config	hostSeconds_total	simSeconds_total	simInsts	cycles	IPC	CPI	L1I_accesses	L1I_misses	L1I_MPKI	L1D_accesses	L1D_misses	L1D_MPKI	L1_total_MPKI	L2_accesses	L2_misses	L2_MPKI	TLB_accesses	TLB_misses	TLB_miss_rate
out_timing	104.13	151.26483	53258688	151808168	0.3508	2.8504	130024908	2247200	42.1941	38756748	1286456	24.1546	66.3489	3994272	777578	14.6	84736422	42609	0.0005
out_o3	223.06	141.679932	44270087	93081844	0.4756	2.1026	15861564	2633676	59.4911	40555553	2547243	57.5387	117.0298	4447368	1045088	23.6071	30448900	167740	0.0055
out_kvm	60.62	151.128328	0	437945373													0	0	

- TLB_misses: *.mmu.itb.rdMisses + *.mmu.itb.wrMisses
- TLB_misses_rate: TLB_misses / (*.mmu.itb.rdAccesses + *.mmu.itb.wrAccesses)

Experimental Result:

Using KVM lets us boot and reach the ROI quickly, then reset stats and continue the ROI on a detailed CPU (Timing/O3). This gave us a short wall-clock time to ROI and trustworthy micro-architectural stats inside the ROI. In contrast, pure KVM is fast end-to-end but can't model the pipeline, queues, or caches. In my CSV, it mostly provides hostSecond, while fields like IPC, cycles are blank or not meaningful. Practically, this means KVM-only is great for speeding through OS work but cannot answer the "why" of performance in ROI. Switching to a detailed CPU at ROI visibly changes the reported statistics. IPC drops relative to the idealized KVM path because the detailed core now performs stalls, cache misses, and TLB behavior. Our ROI rows populate with cycles,

IPC/CPI, L1/L2 MPKI, and TLB miss rate, revealing whether we are memory-bound or compute-bound. In short, KVM + switch is valuable because it keeps running tractable while still giving ground-truth micro-arch metrics where it matters.

Problem 5 (Full-System ROI automation & SE vs FS)

Goal:

Compare SE and FS for the same benchmark/ISA to understand setup overheads and ROI under an OS.

Methodology:

For p5_1(SE), setup with SimpleBoard, Classic cache hierarchy, SingleChannelDDR4, Timing/O3CPU. Running on workload x86-npb-is-size-s-run. Compare p5_1(FE) with p5_2, which use X86Board with the same classic cache and DDR4, running on workload x86-ubuntu-24.04-npb-is-s.

Since the booting part of Linux kernel is not the ROI, I use KVM to accelerate the session, and then switch to a O3/Timing CPU at work begin and dumps the stats.txt at the work end:

```
processor = SimpleSwitchableProcessor(
    starting_core_type=CPUTypes.KVM,
    switch_core_type=detailed_type,
    isa=ISA.X86,
    num_cores=args.cores,
)

sim = Simulator(
    board=board,
    on_exit_event={
        ExitEvent.EXIT: on_exit(),
        ExitEvent.WORKBEGIN: on_work_begin(),
        ExitEvent.WORKEND: on_work_end(),
    },
)
```

Following are the CSV metrics generated by parsing data from stats.txt:

config	outdir	hostSeconds_ROI	simSeconds_ROI	siminsts	cycles	IPC	CPI	L1I_accesses	L1I_misses	L1I_MPKI	L1D_accesses	L1D_misses	L1D_MPKI	L1I_total_MPKI	L2_accesses	L2_misses	L2_MPKI	TLB_accesses	TLB_misses	TLB_miss_rate
fs_o3	log/fs_o3	28.06	0.004612	9317019	13851299	0.6726	1.4867	941559	31444	3.3749	4332959	224984	24.1476	27.5225	249387	48401	5.1940	5556089	21123	0.003802
fs_timing	log/fs_timing	6.36	0.009957	7508469	20892803	0.3594	2.7826	9837286	5555	0.7398	3609044	114631	15.2669	16.0067	176415	32541	4.3339	13465209	13109	0.000974
se_o3	log/se_o3	73.88	0.010507	31414506	31552562	0.9956	1.0044	4335167	1480	0.0471	9579117	382014	12.1604	12.2075	448792	80584	2.5652	14715372	31706	0.002155
se_timing	log/se_timing	22.15	0.02207	31414193	66275318	0.474	2.1097	42238379	890	0.0283	10374598	299815	9.5439	9.5723	449138	70858	2.2556	52613121	31503	0.000599

Experimental Result:

The system-emulation (SE) is faster because it run just the binary, no OS, no drivers, far fewer syscalls, and no context switches. That means fewer interrupts and less kernel

work, so wall-clock time to finish the workload is typically much lower. In my experience, SE completed the ROI noticeably quicker than FS for the same CPU model. The full-system (FS) is more realistic since it includes the kernel and everything that comes with it: system calls, interrupts, I/O, and background services. These effects change the memory hierarchy pressure and pipeline stalls, so micro-architectural stats (IPC/CPI, L1/L2 MPKI, TLB miss rate) in FS often differ from SE. Since the initial operation (boot the Linux kernel) run way too slow in my PC, and those parts are not the ROI, I booted with KVM and switched to a detailed CPU at ROI, dumping and resetting stats at the switch. This preserved realistic OS effects inside the ROI while avoiding a slow detailed boot which are not the observed part.

To know when SE is OK and when FS is necessary, we can use SE for quick, algorithm-level comparisons where OS effects are not critical. And use FS when the workload meaningfully interacts with the OS or I/O, or when we need production-like realism for caches/TLBs under interrupts and kernel code paths. The KVM-to-detailed CPU switch flow is the sweet spot for FS: fast to the ROI, then trustworthy micro-arch statistics.